План текущего модуля транскрибации

Дата: 25 октября 2026 года. Основание — согласованные в чате границы и восемь конфигураций ASR. Базовая версия до изменения: `22ab86697515260ef56e37ded77c32e04d48e8c6`.

Задача и границы

Пользователь записывает вымышленный разговор либо загружает файл, получает полную русскоязычную расшифровку, проверяет и исправляет текст, сохраняет результат. Это первый этап ВКР ОКБ №1, а не готовое автоматическое заполнение медицинской формы. Qwen, выделение жалоб/анамнеза и проверка протокола в текущий учебный модуль не входят.

Готовые модели не обучаются. Эксперимент выбирает подходящую локальную конфигурацию. Старые отчёты с Qwen относятся к прежней версии и не являются доказательствами качества этого модуля.

Требования и проверка

ID	Требование	Приоритет	Проверка
FR01	Явно начать запись, остановить микрофон перед выбором пути, сохранить WAV	Обязательно	Ручная запись; регрессия отмены диалога сохранения
FR02	Загрузить WAV/MP3/M4A/FLAC/OGG/MP4/MKV с аудио, сообщить об ошибке повреждённого файла	Обязательно	Настоящее декодирование в тестах и на пользовательских M4A
FR03	Выбрать одну из восьми локальных конфигураций	Обязательно	Выбор GUI/CLI; реальный запуск каждой конфигурации
FR04	Получить весь текст без обязательного вызова LLM; обработать файл длиннее одного фрагмента	Обязательно	Контроль конца записи и временных границ; реальные прогоны
FR05	Редактировать полный текст, сохраняя исходную расшифровку	Обязательно	Публичная операция Session.edit и сценарий GUI
FR06	Сохранить и восстановить JSON-сеанс; экспортировать текущий текст в UTF-8 TXT	Обязательно	Круговой сценарий с правкой и повторным открытием
FR07	Предупредить перед потерей несохранённого результата; запретить конфликтующие операции	Обязательно	Публичные операции сеанса; ручные проверки
NFR01	Инференс локальный; загрузка моделей — отдельное действие установки	Обязательно	Локальные пути, режим offline и отдельная проверка работы без сети
NFR02	Окно остаётся отзывчивым при обработке	Обязательно	Фоновый worker и инструментальный GUI-сценарий
NFR03	Сохранить версию, настройки, данные, расшифровки, ошибки и условия измерений	Обязательно	Манифесты и журнал эксперимента

Источник требований — пользователь и согласованный сценарий, а не выдуманная клиническая приёмка.

Конфигурации

1. Whisper base через faster-whisper, CPU int8.
2. Whisper small через faster-whisper, CPU int8.
3. Whisper medium через faster-whisper, CPU int8.
4. Vosk small-ru-0.22.
5. Vosk ru-0.42.
6. GigaAM v3 CTC в ONNX.

7. GigaAM v3 RNN-T в ONNX.

8. GigaAM v3 E2E RNN-T в ONNX.

GigaAM выполняется через onnx-asr 0.12.0 / ONNX Runtime CPU с опубликованными int8-экспортами. Это отдельная реализация инференса и экспорт весов; выводы относятся к этим конфигурациям, а не автоматически к любому запуску GigaAM. Реальные источники, ревизии и SHA-256 моделей фиксируются установщиком. Сравнение моделей не подменяет собой сравнение интерфейсов готовых приложений.

Общая обработка и время

Все конфигурации используют одно декодирование PyAV в mono PCM16 16 кГц с переводом в float32 и одинаковые фрагменты длиной не более 20 секунд. Границу длинного фрагмента выбираем в области низкой энергии между 12 и 20 секундами, оставляя хвост не короче 6 секунд; соседние фрагменты покрывают весь звук без пропусков, наложений и удаления пауз. Это решение для ограничений моделей, а не шумоподавление. Внутренний VAD Whisper отключён, чтобы не добавлять отдельный этап только одному кандидату. Фрагментация и настройки фиксируются до основного сравнения. Правило минимального хвоста принято при технической пробе dev01 до просмотра selection.

Восемь CPU-потоков там, где API позволяет это задать; один процесс и одна модель одновременно. Vosk может управлять потоками внутри библиотеки иначе, это ограничение указывается. Whisper: beam_size=5, language=ru, temperature=0, без initial_prompt и передачи эталонов. Декодеры ONNX — greedy. Различия декодеров допустимы как часть заранее выбранных практических конфигураций.

Сначала отдельно измеряется загрузка модели. После прогрева на development выполняются одинаковые повторения selection через Session.process_file — тот же код, что за кнопкой GUI. Время включает чтение/декодирование и распознавание полного файла; ручная правка и действия оператора в скорость модели не входят. Полный пользовательский сценарий проверяется отдельно в GUI. Память — пиковая RSS процесса измерения, в отдельном процессе для каждой модели.

Данные: отбор → очистка → соединение → признаки

Лекция 2.1: слайд 11 — подготовка, 12–13 — разделение и утечки; 17 — оценка данных, модели и продукта.

Получены десять M4A с диктофона, 48 кГц, один канал. Исходники сохранены без изменения; полностью декодированы, точных дубликатов файлов и декодированного звука нет. Для общего набора подготовлены WAV PCM16 mono 16 кГц; звук не обрезался, шумоподавление и усиление не применялись. Автор записей сообщил в чате, что прослушал записи и подтвердил соответствие TXT. Это подтверждение автора, не независимая экспертиза агента или врача.

- dev01/dev02: наладка и проверка настроек.

- dialog01..dialog06: selection — сравнение и выбор.

- control01/control02: итоговая проверка выбранного кандидата. После изменения по контрольным результатам понадобится новый независимый контроль.

Соединение аудио–TXT — один к одному по ID, без размножения строк. Производные записи и её фрагменты остаются в одной группе. Поля манифеста: ID, группа, источник, путь, SHA-256, длительность, частота, каналы, эталон и статус его проверки. Это метаданные для контроля; акустические признаки рассчитывает готовая модель, обучение собственных признаков не проводится.

Утечка предотвращается разделением до настройки: TXT не подаются модели как подсказка, словарь или текст запроса; контроль не используется для выбора настроек. Постобработка не исправляет медицинские термины по известному сценарию. Новая версия данных получает новый манифест. Синтетические прежние наборы не смешиваются с новыми записями человека.

Метрики и выбор

Основная метрика — WER=(замены+пропуски+вставки)/число слов эталона. Общий WER считается по суммарным числителям и знаменателям, не как среднее процентов коротких файлов. Одинаковая нормализация: регистр, ё/е, пунктуация и пробелы. Числовой формат не исправляется по смыслу: отличие цифр от слов отражается в строгом WER и отдельно объясняется при разборе ошибок.

Также: секунды, RTF=время/длительность, пиковая RAM, загрузка модели, сбои и незавершённые файлы. Повторения одного диалога не считаются новыми независимыми записями. Результаты одного читателя в тихих условиях не подтверждают качество в больнице или на других голосах.

Отбор: локальный запуск и завершение полного файла обязательны; кандидаты со сбоями помечаются отдельно. При технической пригодности основной критерий — общий WER на selection; при точном равенстве — медианный RTF, затем пиковая RAM. Если ресурсные показатели делают победителя непригодным для рабочего ПК, это обсуждается как ограничение и решение фиксируется явно, без скрытого сводного балла. Клинический порог WER не выдумывается.

Тестирование и доказательства

Согласованные публичные границы: Session.process_file/edit/save/open/new и пользовательский сценарий GUI. Они уже утверждены в прежнем AGENTS.md и в чате. Внешний ASR/устройство могут заменяться контролируемыми ответами; внутренние Session/Storage/декодирование не мокируются.

Небольшой набор сохраняет проверки загрузки, повреждения, пустого распознавания, повтора после сбоя, правки, сохранения/открытия, несохранённых данных и конфликта. Регрессии настоящего FLOAT WAV и остановки записи остаются. Последовательные red/green циклы подтверждены для обработки без extractor (T01), правки/сохранения (T02), известного примера WER (T12), отказа при дубликатах (T13), фиксации настроек контроля (T14), сохранения результатов прерванного worker (T15). Остальные проверки адаптированы из прежнего набора; тест покрытия длинного аудио T11 написан после реализации и не выдаётся за TDD. Отдельный сценарий настоящего Tkinter проверяет импорт → обработку → правку → TXT/JSON → открытие. Он не является личным ручным тестом студента.

Реальные измерения моделей отделены от быстрых тестов с внешними test doubles. Студенты лично повторяют подготовленную команду и GUI-карточки, записывая исполнителя и фактические результаты. Ручная запись не запускается агентом без явного действия пользователя.

Соответствие ДЗ

1. Разработка — этот модуль и его требования/архитектура.

2. Модель и методы — инкременты с итерациями, Agile-принципы, Kanban-практики, конкретные циклы TDD, CI; примеры из фактической работы, без заявления о полном Scrum.

3. Сравнение — модели по одинаковым данным и измерениям; различие моделей и готовых продуктов обозначено.

4. Методы тестирования — ревью, функциональные, интеграционные, системный сценарий, регрессия и производительность с объяснением выбора.

5. Ручные и автоматические — код проверок и отдельные карточки личного выполнения.

6. Отчёт — среда, версии, набор, результаты, дефекты и ограничения; прежние выводы Qwen не переносятся.

7. Презентацию готовят Юрий и Михаил по проверенным и понятным результатам.

План тестирования TP-ASR-02

Дата: 25.10.2026. Объект: версия 0.2 модуля полной транскрибации. Это рабочий план учебной проверки, а не заключение о готовности медицинского изделия.

Поля плана

Поле	Решение для модуля
Идентификатор	TP-ASR-02; требования FR01–FR07, NFR01–NFR03 из asr-plan.md
Введение	Проверить пользовательскую цепочку от аудио до сохранённого полного текста и измерить различия восьми локальных ASR
Объект	Python/Tkinter-приложение, Session, декодирование, адаптеры ASR, файлы сеанса, скрипт сравнения
Проверяемые функции	Явная запись и остановка, импорт, выбор ASR, весь текст, правка, JSON/TXT, восстановление, ошибки и защита несохранённого результата
Непроверяемые функции	Диагноз, заполнение разделов формы, 1С, телефонный клиент, идентификация пациента; их нет в данном модуле
Подходы	Ревью требований/кода; публичные автоматические проверки; реальный GUI-сценарий; сравнение на проверенных эталонах; личные ручные карточки
Критерии прохождения	Быстрые проверки без ошибок и пропусков; GUI круговой сценарий сохраняет исходник и правки; замеры воспроизводимы. Для кандидата обязательны все запланированные обработки; выбор по правилу в asr-plan.md. Клинический порог WER не установлен
Приостановление	Повреждены или изменились данные/веса/исходники; не подтверждены TXT; закончилась память/место; повторяющийся сбой мешает сценарию
Возобновление	Причина установлена, исправление проверено; новый каталог результата, при изменении данных новый манифест. Прерванный журнал сохраняется
Выходные материалы	tests.log, summary.json; red/green; GUI JSON/TXT; freeze/rows/summary и исходные ответы ASR; карточки ручных проверок; дефекты и итоговый отчёт
Задачи	Подготовка окружения → проверка эталонов → быстрые тесты → dev-пробы → selection → контроль победителя → личная ручная проверка → отчёт
Ресурсы	Windows x64, Python 3.12 с Tk, зависимости lock, CPU/RAM из freeze.json; модели из каталога и манифестов; 10 вымышленных записей. Для M04 требуется настоящий микрофон
Обязанности	Агент реализует и выполняет инструментальные прогоны. Юрий подтвердил TXT по звуку. Юрий и Михаил лично запускают проверки, читают тесты и выполняют карточки. Врачебная приёмка отдельно, не заявлена
Расписание	Этапы выполняются по зависимостям выше. Даты фактических прогонов берутся из UTC-журналов; согласованный срок приёмки заказчиком отсутствует
Риски	Один читатель, короткие записи, неуточнённый диктофон; ошибки чисел/отрицаний; влияние фоновых процессов и нагрева; различия декодеров; время скачивания; ложное впечатление качества по зелёным тестам с подставным ASR
Согласования	Объём подтверждён пользователем в чате. План/инструментальные результаты не являются подписью Михаила, врача или преподавателя

Что проверяют 15 автоматических тестов

Это 15 понятных проверок, а не требование объяснять десятки не связанных с модулем случаев. Подставной внешний ASR выдаёт известный текст, чтобы ошибка модели не мешала проверять управление приложением. Реальные Session, декодирование и запись файлов исполняются. Качество настоящих моделей измеряется другим экспериментом.

ID	Вход и действие	Ожидаемый результат; зачем
T01	WAV + внешний ASR с тремя репликами; process_file	Все реплики в исходном/редактируемом тексте и модель в метаданных; обязательный LLM не вызывается
T02	Получить текст, исправить, сохранить JSON/TXT, открыть	Правка восстановилась, исходник сохранён, TXT совпадает с текущим текстом; проверка интеграции сеанса и файлов
T03	Повреждённый аудиофайл	Понятная AppError; случайные байты не выдаются за запись
T04	Декодируемый файл, ASR не вернул слов	Сообщение о пустом результате; отсутствие слов не выдаётся за успех
T05	Внешний ASR падает один раз, затем отвечает	Повторная обработка возможна; ошибка не блокирует сеанс навсегда
T06	Новый сеанс при несохранённом тексте	Без явного force результат не теряется; проверка защиты данных
T07	ASR ожидает освобождения, вызвана вторая обработка	Вторая операция отклонена; конкурентные результаты не смешиваются
T08	При открытом результате загрузить испорченный JSON	Ошибка формата не заменяет текущий документ
T09	Настоящий IEEE FLOAT WAV	Файл декодируется и доходит до ASR; регрессия прежнего дефекта импорта
T10	Остановить запись, отменить выбор пути; внешний адаптер микрофона	В момент диалога захват уже остановлен, буфер не потерян; регрессия D01
T11	24 секунды PCM с маркером последнего отсчёта	Внешний backend получает все отсчёты по порядку, последний маркер доходит, фрагменты ≤20 секунд; полная обработка длинного файла
T12	Эталон «у меня болит левая рука», ответ «у меня сильно болит правая рука»	1 замена + 1 вставка, 5 слов, WER=2/5=0,4; известный независимый пример расчёта
T13	Одинаковый звук в разных группах, с пометкой дубликата и без неё	Набор отклонён по реальному PCM; не допускаем утечку между группами
T14	Контроль с прежними и затем изменёнными потоками/отпечатками весов	Неизменный вариант принят, изменённые отклонены; контроль относится к действительно выбранной конфигурации
T15	Worker прерван после одной сохранённой строки	Завершённая строка восстановлена, пропуски не исчезли, кандидат не выбран, память помечена неполной

Только T01, T02, T12, T13, T14 и T15 имеют сохранённые последовательные red/green циклы этой разработки. В red-логах T01/T02 зафиксирован прежний несовместимый API (TypeError), T12/T14/T15 - отсутствие нового модуля/функции (ImportError), а T13 - конкретное AssertionError: дубликат был принят. Для объяснения поведенческого TDD особенно нагляден T13. Нельзя утверждать, что во всех шести red уже выполнялись целевые assert. Другие проверки - регрессии/проверки после реализации; их нельзя выдавать за TDD. Расширение признака неполной памяти в T15 добавлено при ревью после реализации.

Уровни и виды

T12 - проверка отдельного вычисления. T01–T09 и T11 - интеграция публичного сеанса с файлами и внешней границей; называть весь модуль одним unit-тестом неверно. T10 и check_gui_scenario.py используют настоящее окно Tkinter, но внешние устройство/модель могут быть подставлены. Прогон check_gui_scenario.py с --model использует настоящую модель и проверяет управляемый GUI путь. Человек отдельно выполняет M01–M08 из manual-tests.md.

Позитивные случаи: правильный файл, правка и восстановление. Негативные: повреждение, пустой ответ, сбой, конфликт, дубликат и изменение условий. Регрессии: прежние дефекты остановки и FLOAT WAV. Производительность: секунды, RTF, RSS в реальном ASR-прогоне. Статическое тестирование - ревью и анализ

типов без запуска пользовательского сценария. Динамическое - выполнение программы. Ручное/автоматическое,уровень и цель - разные классификации, их можно сочетать.

Полноценные нагрузочные, клинические, пентест и исследования удобства с врачами не проводились. Для них нет согласованной среды/пользователей/нагрузки. Малый учебный эксперимент этого не заменяет.

Как выполнить самому

После setup.cmd открыть PowerShell в корне проекта и выполнить:

```
.\.venv\Scripts\python.exe scripts\check.py --out user_data\yuri-checks-01
```

Человек запускает команду, а проверки и сравнение с ожиданиями выполняет программа. Это самостоятельное выполнение автоматизированного тестирования. Сохранить tests.log/summary.json; прочитать один тест, показать вход, вызов, assert и результат. Для Михаила - другое имя папки, собственный фактический запуск. Если есть failure, показать traceback, вход, ожидаемое и фактическое, завести дефект.

Проверка исправления повторяет конкретные шаги дефекта. Регрессия после этого проверяет другие важные функции, которые могло затронуть исправление. Зелёный результат относится к указанной версии и условиям, поскольку другой код, библиотека, устройство или файл меняют то, что проверялось.